



A FORGE & FLOW STUDY GUIDE

Forge Flow Architecture

Plain-English study notes on how the app, the proxy, the database, identity, audit, and the advisor fit together.

Edition
2026-05-19

Reviewed through
2026-05-19 NEXT_WAVE_PLAN snapshot

How To Read This Book

This is a study guide to Forge Flow. It walks through every meaningful part of the system in plain English, explains what each piece does and why it exists, and shows how the pieces fit together. The goal is not to list technology; it is to make the system understandable enough that you can reason about it, ask better questions, and know where each piece belongs.

Each chapter follows the same shape. First there is a short plain-English introduction that gives you the mental model. Then there is a longer prose explanation of the area, where every concept is woven into the narrative rather than listed in a glossary. After that comes a **How it works** walkthrough that shows the runtime path through the system, and finally a **Where it lives** pointer to the relevant code or contract paths so you can dive into the source.

Three status words appear throughout. **Live** means the work is implemented and verified on master or in staging. **Scaffolded** means the foundation exists but a downstream consumer or workflow is still queued. **Planned** means the contract or phase doc describes it, but the code does not yet exist.

A reasonable reading order is to start with the whole-system picture and the quick walkthroughs, then read the product loop so the operational concepts make sense, then read app, SQLite, proxy, DNS, auth, and Postgres as the core runtime path. From there, RLS, permissions, service principals, and audit form the safety path. Events, usage, advisor, graph, integrations, shared state, and workflow form the expansion path. Deployment, testing, status, the glossary, and the source-file list are reference material you can dip into as needed.

Contents

PART I, THE WHOLE SYSTEM

- 1 Whole System In One Picture
- 2 Quick Walkthroughs Before The Details
- 3 The Product Loop
- 4 Repository Map

PART II, THE RUNTIME PATH

- 5 The App Runtime And UI
- 6 SQLite, Local App Database
- 7 Postgres, Shared Server Truth
- 8 The Proxy And API Layer
- 9 DNS, Edge, And Cloud Run

PART III, THE SAFETY PATH

- 10 Tenant Safety And Row Level Security
- 11 Authentication
- 12 MFA And Admin Reset
- 13 Permissions, Roles, And Admin
- 14 Service Principals
- 15 Audit And Compliance

PART IV, THE EXPANSION PATH

- 16 Eventing And Real Time
- 17 Usage, Caps, And Rollups
- 18 The Advisor And AI Layer
- 19 The Graph Layer

20 Live Integrations

21 Shared State Roadmap

22 The Operations Console

23 The Workflow Platform

PART V, OPERATIONS & REFERENCE

24 Deployment, Secrets, And Environments

25 Testing And Gates

26 Current Status And Known Gaps

G Quick Glossary

S Source Documents And Code Read

Whole System In One Picture

The eight pieces that make Forge Flow, and how a request travels through them.

MENTAL MODEL

Forge Flow helps an operator understand what the plan was, what is happening right now, whether labor is on track, whether demand has changed, what happened historically, and what the system recommends next. The app on a device is only one part of that system.

The bigger picture includes a server-side proxy, a shared cloud database, an identity service, audit logs, DNS, edge protection, an AI retrieval layer, and a future workflow automation tier. Each of those pieces has a single job, a single deployment boundary, and a single trust posture, and the system works because the handoffs between them are explicit.

The Core System Map

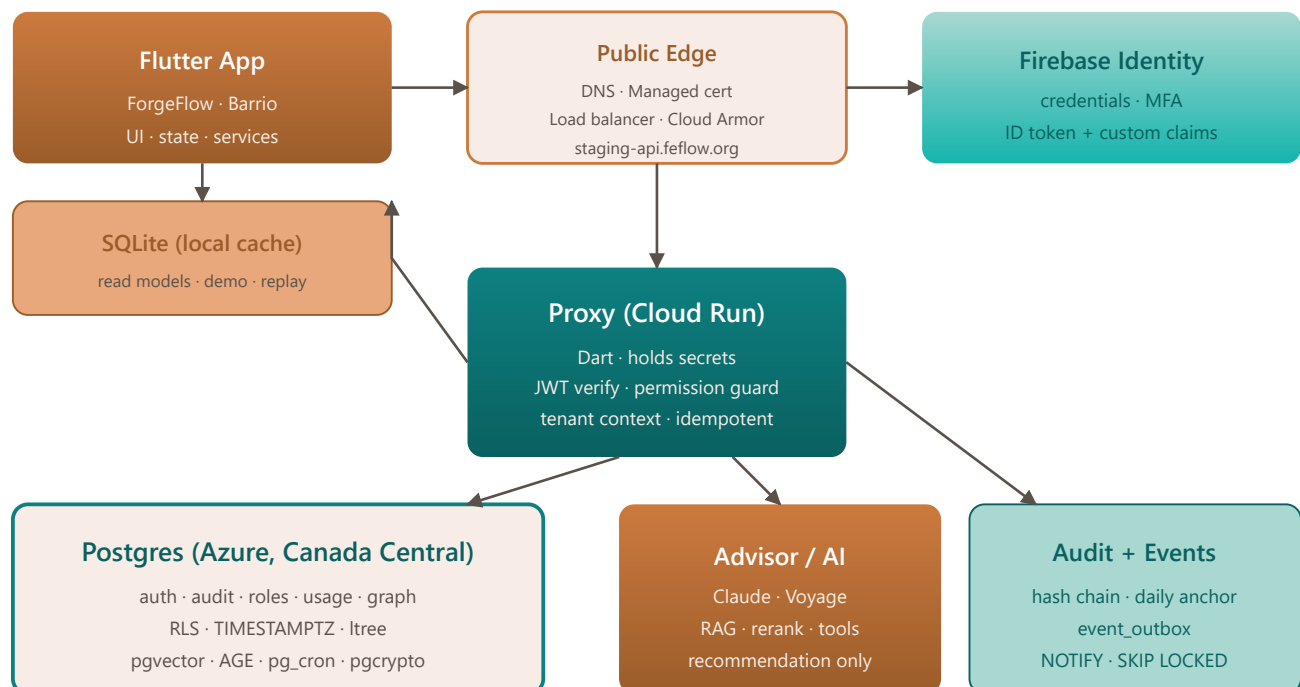


Figure 1.1. The thin Flutter client sits above a fast local cache. Public requests enter through the managed edge, identity comes from Firebase, and every protected action flows through the proxy into Postgres, the advisor, and the audit/event spine.

The Flutter app is the user-facing surface, written in Dart, running on mobile, web, and desktop, in two flavors today: the main **ForgeFlow** product and the branded **Barrio** flavor. The app holds no privileged secrets. It reads cached state from a small local SQLite database, asks Firebase Identity Platform to verify credentials, and asks the server-side proxy to perform anything that involves shared data, vendor credentials, or privileged action.

The **proxy** is a Dart server living under `tool/advisor_proxy/`. It is the only place in the system that holds database passwords, vendor API keys, and the Firebase admin surface, and it is the only place that talks directly to Postgres. Every protected action the app needs (sign-in handoff, MFA management, role lookups, admin operations, advisor calls) flows through the proxy. It deploys as a container to Google **Cloud Run**, which scales it without us managing servers.

The **shared database** is **Azure Database for PostgreSQL Flexible Server, version 16, in Canada Central**. Postgres holds everything that must be correct across users, devices, operators, and audits: auth sessions, audit logs, operator and location records, roles and permissions, usage and caps, advisor evidence, graph relationships, and the rollup tables that feed dashboards. The proxy talks to Postgres through a small set of safe abstractions; nothing else does.

Firebase Identity Platform sits beside the proxy as the credential authority. It validates passwords, manages multi-factor enrollment, issues signed ID tokens, and holds the custom claims that map a Firebase user back to a Forge Flow operator, location, and role. Forge Flow trusts Firebase to say "this is a real authenticated person" and then layers its own business identity on top.

At the public edge, traffic enters through **DNS** (for example `staging-api.feflow.org`), terminates HTTPS at a **Google managed load balancer** with a **Google managed certificate**, passes through **Cloud Armor** for edge filtering, and is forwarded to Cloud Run via a **serverless network endpoint group**. The **advisor**: Forge Flow's AI surface, sits above this stack and uses model providers, embeddings, vector search, graph traversal, and SQL evidence to ground its recommendations rather than answering from generic chatbot memory.

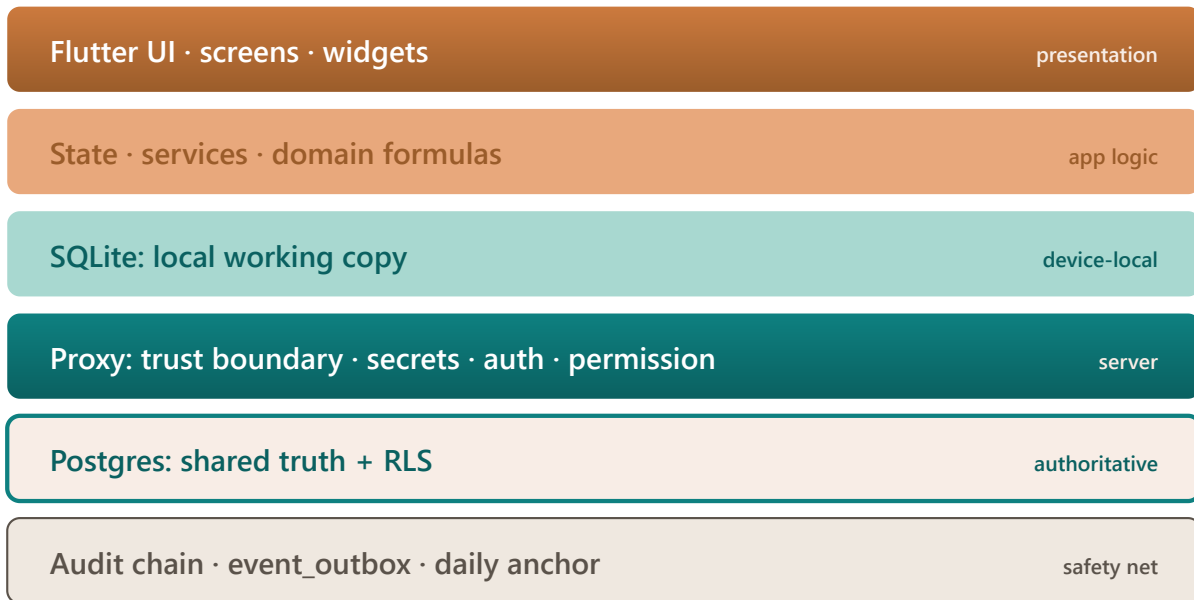


Figure 1.2. The layered stack. UI sits at the top; trust deepens as you move down toward the audit and event substrate.

How It Works

```
User opens Flutter app
-> app reads local SQLite cache
-> user signs in through Firebase
-> app calls proxy for protected operations
-> proxy verifies identity and permissions
-> proxy reads/writes Azure Postgres
-> Postgres applies tenant safety and audit rules
-> app receives safe results
```

For public staging traffic the path is:

```
https://staging-api.feflow.org
-> DNS resolves the hostname
-> Google load balancer receives traffic
-> Google managed certificate terminates HTTPS
-> Cloud Armor evaluates edge rules
-> Cloud Run runs the proxy
-> proxy answers the route
```

WHERE IT LIVES

The app lives under `lib/`. The proxy lives under `tool/advisor_proxy/`. Local persistence lives under `lib/infrastructure/persistence/sqlite/` and server persistence under `lib/infrastructure/persistence/postgres/`. Database migrations are in `db/migrations/`. Architecture contracts, phase plans, and runbooks live in `docs/`, `docs/contracts/`, `docs/phases/`, and `runbooks/` (a single unified home as of the 2026-05-16 Wave 4 consolidation).

Quick Walkthroughs Before The Details

Four concrete journeys that make the moving parts feel real.

Before the detailed chapters go deeper, four short walkthroughs make the system concrete. Each one is a single user-facing event, traced step by step through the layers introduced in chapter 1.

A. User Login

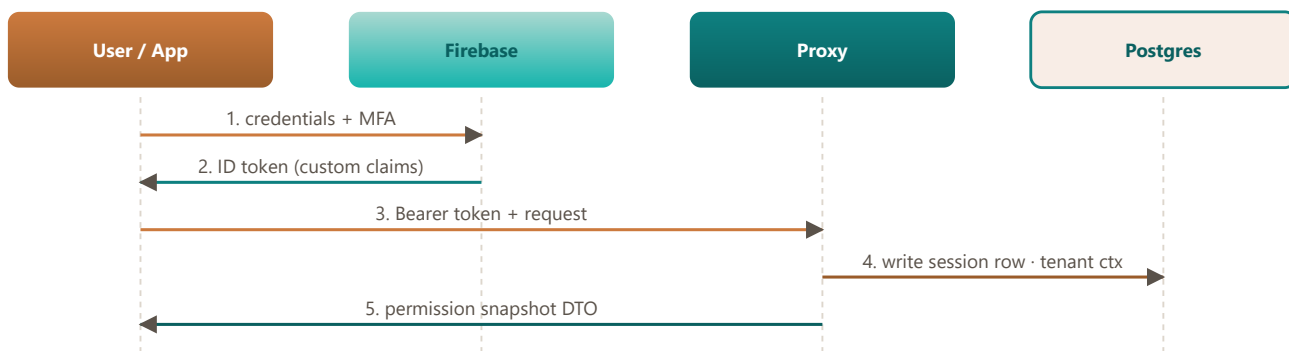
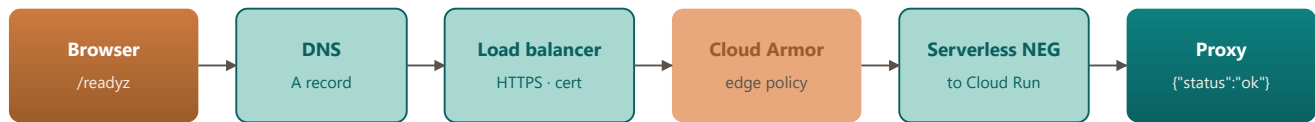


Figure 2.1. The login handshake: Firebase proves the person, the proxy records the session and resolves Forge Flow business identity.

A user enters email and password. Firebase validates the credentials and returns a signed ID token, and the app passes that token to the proxy. The proxy checks that the required custom claims are present, resolves the operator, location, and user context, writes a row to the auth session ledger in Postgres, and returns a permission snapshot. The app stores a secure session envelope locally and unlocks the protected surfaces. In one sentence: Firebase proves the person, and Forge Flow records the session and decides what business access that person has.

B. Staging Health Check



DNS resolves the friendly name to Google's reserved edge IP (34.54.204.29).

Figure 2.2. The staging health probe path, from friendly hostname to a healthy Cloud Run response.

A browser requests `https://staging-api.feflow.org/readyz`. DNS resolves the name to Google's reserved load-balancer IP, the load balancer terminates HTTPS using the Google-managed certificate, Cloud Armor evaluates its edge policy, the serverless NEG forwards the request to the `forge-flow-staging-proxy` Cloud Run service, and the Dart proxy returns `{"status":"ok"}`.

C. Advisor Answer

A user asks an operational question. A classifier picks the route (docs, SQL, graph, or mixed). Evidence is retrieved from the chosen sources, Voyage reranks the candidate evidence so the most useful pieces come first, Claude synthesizes a final answer that cites its sources, and the app shows the recommendation alongside the supporting context.

D. Future Shared-State Update

A manager updates an official schedule. The app sends the mutation to the proxy. The proxy verifies the JWT and the permission, the Postgres write runs inside a tenant-scoped transaction, an `event_outbox` row is written next to it, and a worker fans the event out so other devices know their local cache is stale.

The Product Loop

Twelve layers from raw evidence to learned recommendations.

MENTAL MODEL

The product is built around a loop: what happened before, what good should look like, what we expect this week, what we planned, what is happening now, what actually happened, and what we learn. Live data is messy, so Forge Flow keeps those meanings separate.

Sales change, labor changes, reservations move, a manager adjusts a schedule. If planned, live, and historical truths get mixed together the system becomes hard to trust. Forge Flow's canonical architecture, recorded in [docs/contracts/core_app_architecture.md](#), organizes the product into twelve named layers so each kind of truth has its own home.

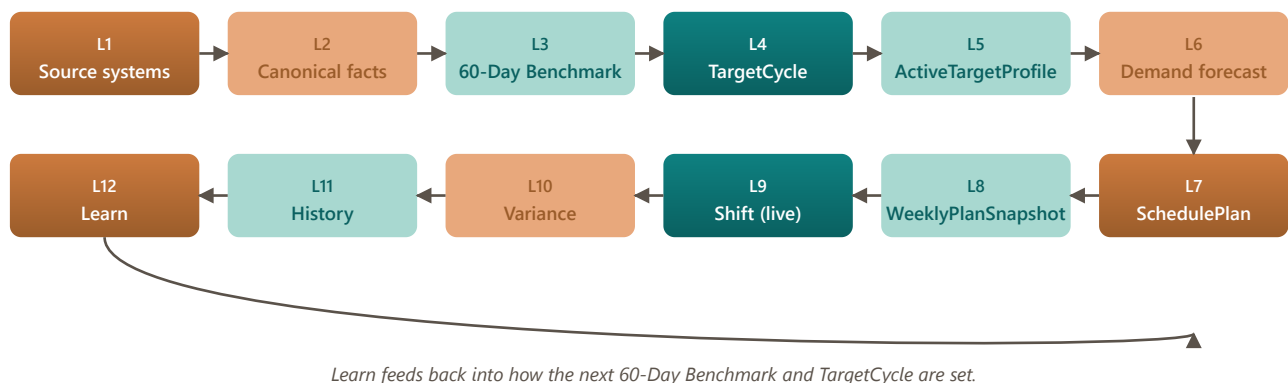


Figure 3.1. The twelve layers of the canonical product loop. Each layer has its own home; provenance rules forbid collapsing one into another.

The first layer is **source systems** (POS, labor, reservations, accounting, banking) that produce raw evidence. The second is **canonical operational facts**: every vendor's exports, webhooks, and API payloads get translated into Forge Flow's own normalized shape so screens and advisor logic never see vendor-specific mess. The third is the **60-Day Benchmark**, the trusted history window that defines what "good" looks like for a comparable service. The fourth is the **TargetCycle**, a locked benchmark window that prevents the standard from drifting after a plan is made, and the fifth is the **ActiveTargetProfile**, the current target settings that the screens and formulas read against.

The sixth layer is the **DemandForecastContext**, which combines reservations, walk-ins, and history into expected demand so staffing decisions are not based on averages alone. The seventh is the **SchedulePlan**, the operator's intended staffing, which then locks into the eighth layer, the **WeeklyPlanSnapshot**: the version of the plan that existed at the moment it was approved, so future variance comparisons are fair.

The ninth layer is **Shift**, the live or whole-day view that shows what is happening right now. The tenth is **Variance**, the gap between plan and actual that turns raw numbers into an on-track / off-track signal. The eleventh is **History**, the closed truth that completed periods leave behind, and the twelfth is **Learn**, the teaching layer that uses repeated closed evidence to explain patterns and recommend coaching.

The active feature line on top of these layers as of 2026-05-19 is **Per-Daypart Targets V1**, recorded in `docs/phases/per_daypart_targets_v1/per_daypart_targets_v1_plan.md`. Per-daypart targets push the existing whole-day TargetCycle down to a daypart grain so lunch and dinner can carry distinct expected covers, labor, and verdict thresholds. The plan consolidates 44 surface-coverage gaps from the mobile walkthrough and ships in **seven slices** (Slice 0 through Slice 6); as of the 2026-05-19 status snapshot, Slices 0 through 5 have landed and Slice 6 is in flight.

How It Works

```
POS + labor + reservation systems
-> canonical operational facts
-> 60-day benchmark snapshot
-> TargetCycle + DemandForecastContext
-> SchedulePlan
-> WeeklyPlanSnapshot
-> Shift
-> Variance
-> History
-> Learn
```

WHERE IT LIVES

The canonical architecture and its binding rules live in `docs/contracts/core_app_architecture.md`. App state and runtime services live under `lib/state/`, `lib/services/`, and `lib/domain/`. The active feature plan is `docs/phases/per_daypart_targets_v1/per_daypart_targets_v1_plan.md`.

Repository Map

Which folder owns which responsibility, and why that matters.

The repository is split by responsibility so that UI code, database code, cloud code, planning notes, and tests each have a clear home. When you make a change, the first question is which layer it touches: UI, app state, local data, server API, database schema, cloud deployment, or documentation.

The Layout

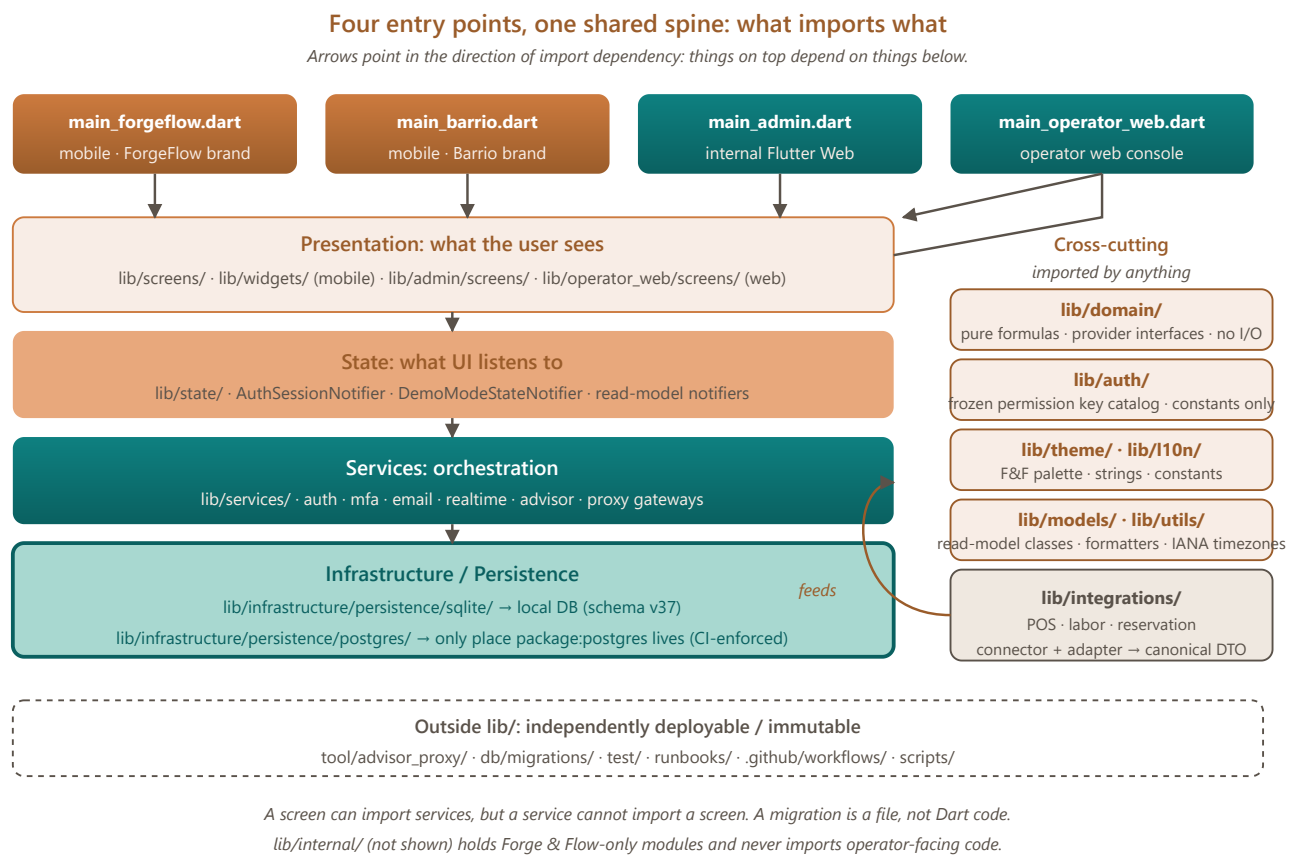


Figure 4.1. Four app entry points all funnel down through the same four-layer Dart stack (presentation → state → services → infrastructure). The right column holds cross-cutting modules every layer is allowed to import; `lib/integrations/` hooks in at services. Arrows point in the import direction.

The main Dart app lives under `lib/`. `lib/domain/` holds pure formulas and provider interfaces such as `llm_provider.dart`, `embedding_provider.dart`, and `rerank_provider.dart` under `lib/domain/services/`. `lib/services/` holds runtime services that coordinate auth, providers, gateways, and app operations. `lib/state/` holds the notifiers and read models that the UI listens to. `lib/dev/` contains demo-only and developer-only code, `lib/auth/` holds the frozen permission key catalog, and `lib/internal/` holds Forge & Flow internal-only modules that should never be touched by operator-facing surfaces.

Persistence is split in two: `lib/infrastructure/persistence/sqlite/` defines the local schema and DAOs, while `lib/infrastructure/persistence/postgres/` holds the only place in the codebase where raw `package:postgres` imports are allowed, CI lint enforces that boundary. Three large surfaces sit alongside the main app: `lib/admin/` (internal admin console), `lib/operator_web/` (operator web console), and `lib/integrations/` (vendor integrations for labor, POS, and reservation data).

A second group of `lib/` subdirectories supports the UI without owning runtime behavior. `lib/screens/` holds the actual screen widgets that the routes render into. `lib/widgets/` holds the reusable widget primitives shared across screens. `lib/models/` holds plain data classes (read models like `shift_record` and `week_record`, plus the teaching summary shapes used in chapter 18) that flow between services and screens. `lib/theme/` holds `app_theme.dart`, which is the single source of the Forge & Flow brand palette (Sunset, Peacock, Jade, Ocean, Red Sand, Shell) referenced throughout the product and this book. `lib/l10n/` holds the localization entries, and `lib/utils/` holds small cross-cutting helpers like `formatters.dart` and the IANA timezone resolver used by chapter 20's integration timestamps.

Outside `lib/`, the proxy lives under `tool/advisor_proxy/` as an independently deployable Dart server. Database schema history lives under `db/migrations/`. Tests live under `test/`. Architecture contracts live under `docs/contracts/`, active phase plans under `docs/phases/`, and operational procedures under `runbooks/`, which the 2026-05-16 Wave 4 consolidation merged into a single unified home. CI workflows live under `.github/workflows/`, and helper scripts (including the staging proxy deploy script) live under `scripts/`.

How It Works

```
docs/contract or phase plan
-> db/migrations if schema changes
-> lib/infrastructure if persistence changes
-> lib/services if orchestration changes
-> lib/state if app state changes
-> lib/screens/widgets if UI changes
-> test/ coverage
```

WHERE IT LIVES

The repo index is `docs/README.md` . Root status is `PROJECT_TRACKER.md` . Repo rules and the authority order are `CLAUDE.md` . The forward plan is `docs/_indices/NEXT_WAVE_PLAN.md` .

The App Runtime And UI

Flavors, entrypoints, bootstrap, and the auth gate that protects the dashboard.

The app is the part users see, with screens for the current shift, variance, plans, benchmarks, settings, and notifications. It ships in two flavors today, **ForgeFlow** (the main product, launched from `lib/main_forgeflow.dart`) and **Barrio** (a branded/demo variant, launched from `lib/main_barrio.dart`), and both flavors share the same architectural backbone. Two additional entrypoints exist for non-app surfaces: `lib/main_admin.dart` boots the internal admin console, and `lib/main_operator_web.dart` boots the operator web console.

The Runtime Path

The visible app needs a reliable startup path before any protected screen renders. The flavor entrypoint is the first file that runs; it chooses which runtime bindings to install. From there, `bootstrapAndRunApp` performs the work of starting the app: it checks whether the Firebase auth runtime is enabled, builds the dependency tree, hydrates local SQLite-backed state, creates the `AuthSessionNotifier`, attempts to rehydrate any secure session envelope already on the device, and only then constructs the provider tree that feeds the widget tree.

What every gate must pass before the dashboard renders

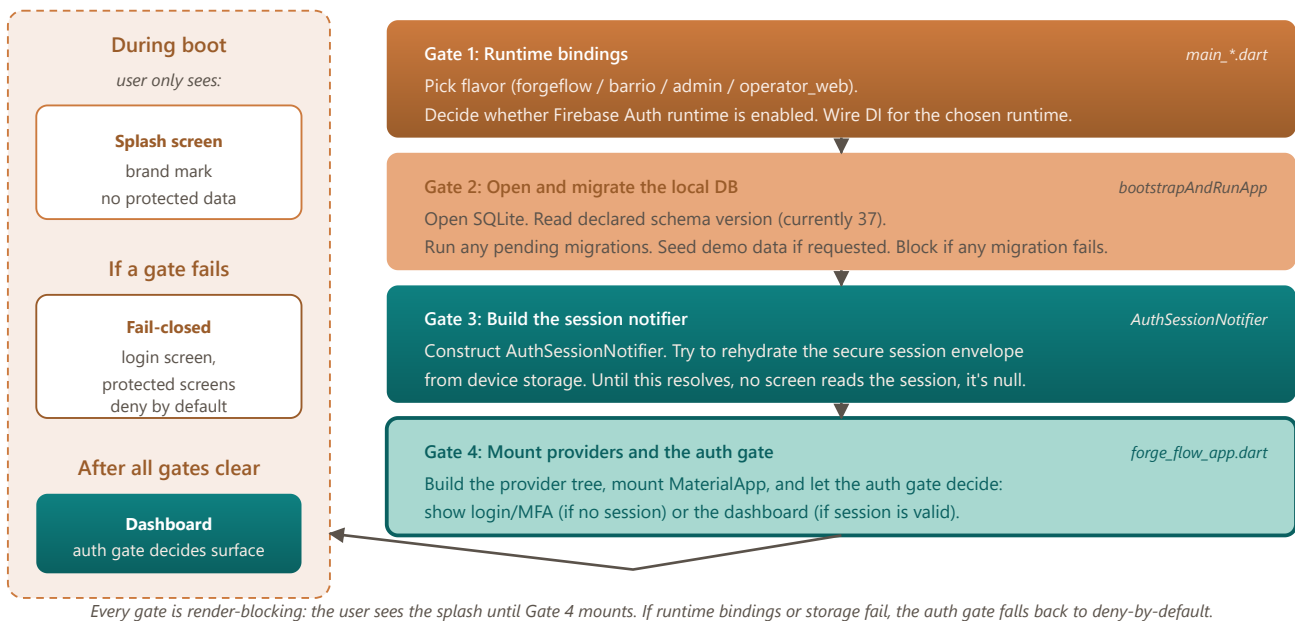


Figure 5.1. The app's boot is four render-blocking gates, not a parade of dots. The left column shows what the user sees while each gate resolves; the right shows what each gate has to satisfy before the next one runs. Gate 3 is the reason the dashboard never flashes before the session is known.

Flutter is the UI framework that makes the app portable across mobile, web, and desktop, and Dart is the shared language across the app and the proxy. The `provider` package wires dependencies and notifiers into the widget tree. **Notifiers** are state objects that emit changes; screens listen and rebuild when state updates. The `MaterialApp` shell owns routing, theme, and global navigation, and an **auth gate** sits between the shell and the protected surfaces so a logged-out user sees the login or MFA flow instead of the dashboard.

How It Works

```
main_forgeflow.dart or main_barrio.dart
-> checks whether Firebase auth runtime is enabled
-> builds runtime bindings
-> bootstrapAndRunApp
-> hydrate local SQLite-backed state
-> create AuthSessionNotifier
-> try to rehydrate secure session
-> build provider tree
-> show the app shell
```

WHERE IT LIVES

Entrypoints: `lib/main_forgeflow.dart` , `lib/main_barrio.dart` . Bootstrap:
`lib/forge_flow_bootstrap.dart` . App shell: `lib/forge_flow_app.dart` . Auth state:
`lib/state/auth_session_notifier.dart` .

SQLite, Local App Database

The fast local working copy that keeps the UI responsive, and what it must *not* own.

MENTAL MODEL

SQLite is the app's local working copy, fast, in-process, used for demo data, replay scenarios, local app state, cached read models, and offline-friendly reads. Anything that must be correct across users, devices, operators, or audits belongs in Postgres.

The local database layer is built on `sqflite` for mobile builds and `sqflite_common_ffi` for desktop and test environments. A monotonically increasing schema version (currently `37`, declared in `lib/infrastructure/persistence/sqlite/sqlite_database.dart`) tells the app which migrations to run when an existing install opens. On first launch or whenever demo data is requested, seed data populates the local database with a known starting restaurant and replay fixtures, and a mock-replay state machine drives sample business days through the system so screens have realistic operational data to render against even before live integrations are wired up.

Above the raw storage, the layer defines screen-friendly read models, pre-shaped row types that the UI can render directly, and caches operational data so that complex dashboards do not have to recalculate from raw facts on every frame.

DEMO MODE IS A WRITER-SIDE SWITCH

Forge Flow's second Hard Promise says demo mode is a *writer-side* switch, not a reader-side fork: demo data lives in the same standard SQLite tables as production data, under a single `DemoScope.restaurantId = 'demo_restaurant_001'`. The reader code never branches on `kDemoMode`; only writers do. The May 2026 surface `lib/screens/settings/settings_demo_live_switch.dart` adds a runtime user-facing demo↔live control that reads `DemoModeStateNotifier` and calls the proxy to flip per-(operator, location, category) `demo_mode_state` rows in Postgres without a rebuild.

SQLite vs Postgres at a glance

Dimension	SQLite (this chapter)	Postgres (next chapter)
Trust level	Local working copy	Shared system of record
Audience	One device / one user	All users, devices, operators
Lifetime	Until app data is cleared	Durable, backed up
Security	Device-protected storage	RLS · TLS · Secret Manager
Best at	Read models, cache, demo	Audit, roles, shared truth
Worst at	Cross-device coordination	Sub-millisecond UI reads

How It Works

```
app starts
-> opens local SQLite database
-> checks schema version
-> runs needed migrations
-> seeds demo/replay state if needed
-> services read local records
-> state notifiers produce screen data
-> UI displays the result
```

WHERE IT LIVES

SQLite code: `lib/infrastructure/persistence/sqlite/` . Desktop/dev database file: `forge_flow_v2.db` .
Current schema version: `37` . Demo-mode contract: `docs/contracts/demo_mode_contract.md` .

Postgres, Shared Server Truth

The authoritative database for everything that has to be correct across users.

Postgres is the official shared system of record. SQLite is the local working copy; Postgres is the source of truth that must work across users, devices, operators, audits, admin actions, advisor systems, and future automation. The app should never bypass Postgres for shared or sensitive data.

The Managed Platform

Forge Flow runs on **Azure Database for PostgreSQL Flexible Server**, version 16, in the **Canada Central** region. Azure handles hosting, backups, and platform maintenance. The choice of PostgreSQL 16 fixes the available syntax, the supported extensions, and the baseline behavior that migrations target.

The Access Abstractions

Application code never talks directly to a Postgres connection. Repositories call a `PostgresExecutor` to run SQL, a `PostgresTransaction` to bundle several operations so they succeed or fail together, and a `PostgresPool` to reuse connections rather than opening a new socket per query. A `TenantContext` value carries the current operator, location, and user identifiers into every protected transaction. The `OperatorScopedRepository` base class enforces that operator-owned data access always runs inside tenant context. The raw `package:postgres` driver is held behind an adapter and is the only place in the codebase where direct database sockets are allowed, a CI lint enforces that no other file imports it.

Important Extensions

What our extensions let Postgres do that vanilla SQL can't

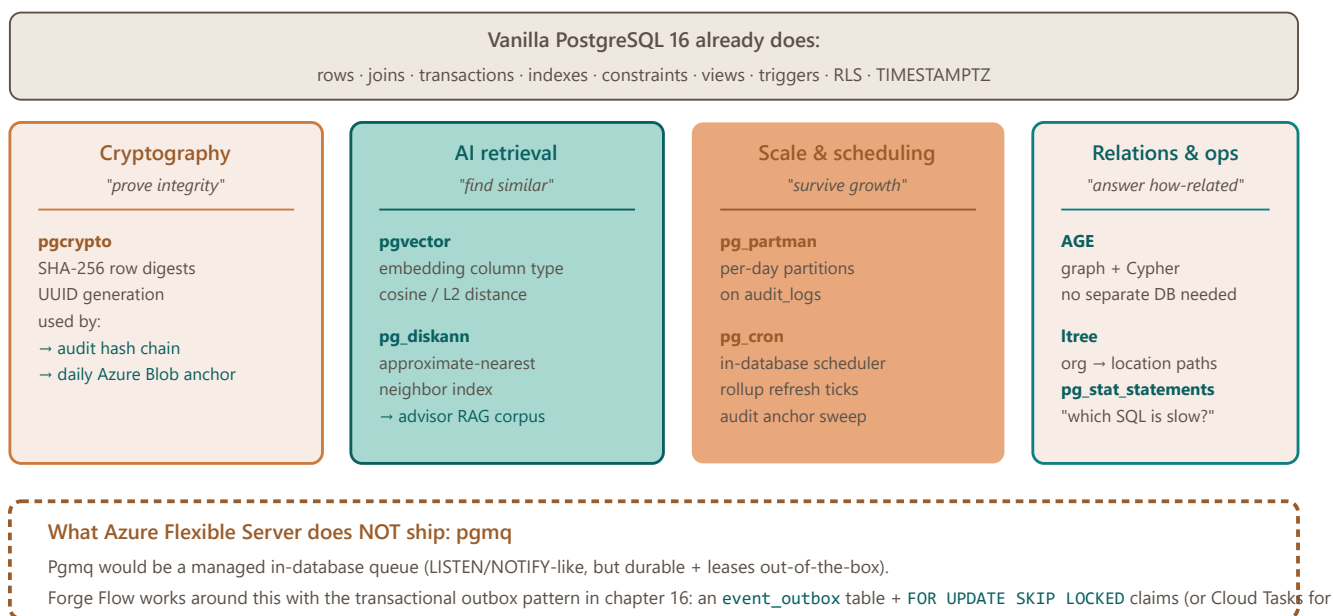


Figure 7.1. Each column is a capability our product depends on. Each extension says what it adds, plus which Forge Flow feature calls it. The dashed box at the bottom is the extension Azure doesn't ship, and what we do instead.

Hierarchy-scoped settings use the business → org-unit → location tree directly: business-level values inherit downward, org-unit values override their ancestors, and location values override everything above them. Any admin, operator, proxy, mobile, or migration work that exposes settings must preserve that rule and show which scope produced the effective value.

Important Execution Patterns

Most database work runs through `runInTenantContext`, which starts a transaction and sets the operator, location, and user identifiers as transaction-local Postgres variables. Because these settings are transaction-local rather than session-global, tenant context cannot leak across requests on a reused pooled connection. Maintenance work that is not tied to a single user action runs through `runAsSystem` with an explicit reason. Outside these two paths, raw driver usage is prohibited.

How It Works

```
proxy receives request
-> verifies identity
-> builds TenantContext
-> repository calls runInTenantContext
-> Postgres transaction starts
-> tenant variables are set locally for this transaction
-> SQL runs
-> RLS filters allowed rows
-> transaction commits or rolls back
```

WHERE IT LIVES

Postgres code: `lib/infrastructure/persistence/postgres/`. Migrations: `db/migrations/`. Production apply guidance: `runbooks/phase_9_production1_migration_apply_runbook.md`.

The Proxy And API Layer

The server-side front desk that holds the secrets and enforces the rules.

MENTAL MODEL

The proxy is the secure server-side front desk. The app does not hold database passwords, vendor secrets, admin credentials, or privileged service keys, the proxy does. When the app needs protected work done, it asks the proxy, and the proxy verifies who the caller is, which operator and location they belong to, what permissions they hold, and whether the request is allowed before it touches the database or any external service.

Routes, Tokens, And Trust

Each capability the app needs is a versioned route under `/v1/...`; breaking changes get a new prefix like `/v2/...`, and old paths stay live until they are explicitly deprecated. Routes use standard HTTP verbs and exchange DTOs with typed fields. Every write is idempotent: the client carries an idempotency key, and the proxy records that key in a `proxy_requests` table with a UNIQUE constraint so that retries cannot accidentally double-charge or double-act.

Identity arrives as a Firebase-issued JWT in the `Authorization: Bearer` header. The proxy verifies the signature, reads the required custom claims (`operator_id`, optional `user_id`, optional `location_id`, role and admin flags, role-version fields), and rejects the request fail-closed if anything is missing. Permission guards then check that the resolved identity is actually allowed to perform this specific action before any data-changing code runs.

The proxy is a Dart executable that runs as a container on **Cloud Run**. The staging deployment is named `forge-flow-staging-proxy`. There is no parallel API stack: every AI surface, every admin action, and every shared-state mutation plugs into the same proxy plus the provider abstractions, counter and cap logic, and feature flags introduced in Phase 11a.10.

Anatomy of a single proxy request

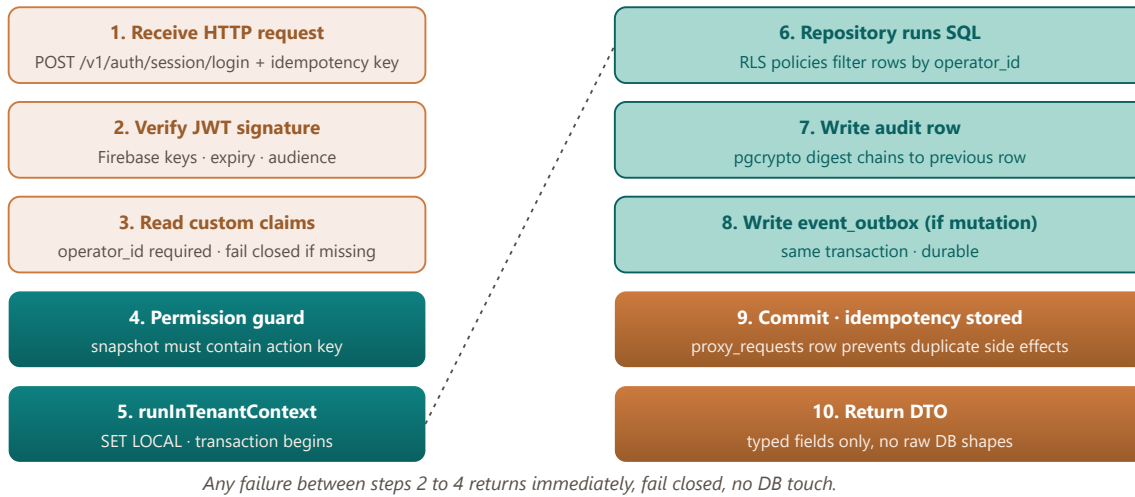


Figure 8.1. Ten stages of a single proxy request. Steps 4 to 5 are the trust boundary; steps 6 to 9 are the durability boundary.

How It Works

```
Flutter app calls /v1/auth/permissions/snapshot
-> proxy reads Bearer token
-> proxy verifies Firebase JWT
-> proxy derives OperatorContext
-> proxy checks permission rules
-> proxy opens tenant-scoped Postgres transaction
-> repository reads allowed data
-> proxy returns safe response DTO
```

WHERE IT LIVES

Proxy entrypoint: `tool/advisor_proxy/main.dart` . Wiring: `tool/advisor_proxy/proxy_bootstrap.dart` .
Route handlers: `tool/advisor_proxy/advisor_proxy.dart` . Container build: `Dockerfile` . Deploy script:
`scripts/deploy_staging_proxy.ps1` .

DNS, Edge, And Cloud Run

From a friendly hostname to a running container.

Apps and users do not call a raw server process; they call a friendly name. For staging that name is `staging-api.feflow.org`. DNS translates that name into a network destination, Google receives the traffic, HTTPS and edge rules are applied, and the request is forwarded to the Cloud Run proxy. The naming and edge layer is what makes infrastructure changeable behind the scenes without forcing client updates.

How A Public Request Travels

A **hostname** like `staging-api.feflow.org` is the stable client-facing identifier. **DNS** translates hostnames into destinations. An **A record** points directly to an IPv4 address, Google's reserved load-balancer IP `34.54.204.29`, and a **CNAME** points one hostname at another. The current staging fix is an A record to the reserved Google edge IP; CNAME is the pattern to reach for when a provider owns a target hostname whose underlying IPs may change.

HTTPS trust comes from a **Google-managed certificate** issued for the hostname once DNS is publicly visible. The **load balancer** is the public entry point that terminates HTTPS and routes traffic to the backend. Before traffic reaches the backend, **Cloud Armor** evaluates an edge policy that logs or blocks suspicious requests. A **serverless network endpoint group** (serverless NEG) connects the load balancer to the **Cloud Run service**: `forge-flow-staging-proxy` for staging.

How It Works

```
https://staging-api.feflow.org/readyz
-> DNS A record
-> 34.54.204.29
-> Google HTTP(S) load balancer
-> Google managed certificate
-> Cloud Armor/reCAPTCHA policy path
-> serverless NEG
-> forge-flow-staging-proxy Cloud Run service
-> Dart proxy route
-> {"status":"ok"}
```

WHERE IT LIVES

Edge status: `PROJECT_TRACKER.md` . Deploy script: `scripts/deploy_staging_proxy.ps1` . Proxy health routes: `tool/advisor_proxy/advisor_proxy.dart` .

Tenant Safety And Row Level Security

Two layers, repository discipline first, RLS as the safety net.

MENTAL MODEL

Forge Flow is multi-tenant: many operators and many locations can live in the same database. The rule is simple, Operator A must not see Operator B's data. The app and proxy code pass the correct context on every protected request; Postgres Row Level Security blocks rows that fall outside that context even if a query forgets to filter.

Identity And Policies

Every protected database operation carries three identifiers: `operator_id` labels which customer/operator owns the work, `location_id` labels which restaurant location it belongs to, and `user_id` labels who is performing the action. That identity context travels with the transaction so the app, the proxy, the audit log, and the RLS policies are all talking about the same tenant and actor.

Row Level Security is a Postgres feature that hides rows that do not match the current context. Policies are written using a small set of approved **wrapper functions** (for example, `app_current_operator_id()`) because the wrappers are `STABLE LEAKPROOF PARALLEL SAFE`. A CI lint at `tool/rls_policy_lint.dart` enforces that policies never read bare session settings.

Tenant context is set with `set_config(..., true)` so that the current operator, location, and user are recorded only for the duration of the current transaction. This is critical because the proxy reuses pooled database connections. Every operator-scoped table is also indexed with a tenant-leading index (typically a B-tree starting with `operator_id` or `(operator_id, location_id)`). Operator-scoped tables store timestamps as `TIMESTAMPTZ` (UTC) plus a denormalized `business_date` column, so a late-night close at 1 AM still belongs to the correct operational day.

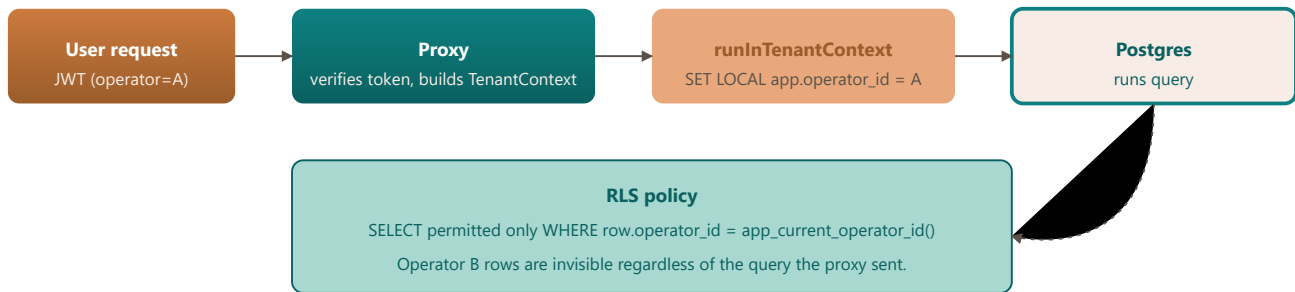


Figure 10.1. Repository discipline carries operator context into the transaction; RLS enforces it at the row layer as a safety net.

How It Works

```
verified user belongs to operator X
-> proxy creates TenantContext(operator X)
-> transaction sets app.operator_id = X locally
-> query asks for records
-> RLS policy checks current operator
-> Postgres returns only rows for operator X
```

The non-negotiable rule is: never use global session settings for tenant context. Use transaction-local settings so context cannot leak between requests on a reused connection.

WHERE IT LIVES

RLS wrappers: `db/migrations/202604280000_phase_9_0sigma_b_rls_wrappers.sql` . Policy rewrites: `db/migrations/202604280001_phase_9_0sigma_b_rewrite_existing_policies.sql` . RLS shape lint: `tool/rls_policy_lint.dart` . Contract: `docs/contracts/hardening_rls_and_repository_pattern_contract.md` .

Authentication

Firebase proves the person; Forge Flow records the business identity.

Authentication answers "Who are you?" Firebase handles the credential side: it confirms the password and MFA, says which Firebase user is signed in, and issues a legitimate signed ID token. Forge Flow then takes that proof and adds business meaning: which operator the user belongs to, which location they can act in, what roles and permissions they hold, and which session should be recorded.

Identity Handoff And Sessions

Firebase Identity Platform is the credential authority. It manages the email-password sign-in flow, the MFA factor enrollment, and the issuance of ID tokens, which are JSON Web Tokens signed by Firebase. Each Firebase user has a stable UID, and Forge Flow's user records reference that UID so that the business identity in Postgres is permanently linked to the credential identity in Firebase.

What turns a Firebase token into a Forge Flow session is the set of **custom claims** baked into the token. These claims carry the `operator_id`, the optional `user_id` and `location_id`, the role and admin flags, and the role version fields that let the proxy detect when a user's permissions have changed since the token was issued. If any required claim is missing, the proxy and the app both fail closed and refuse the request rather than guess.

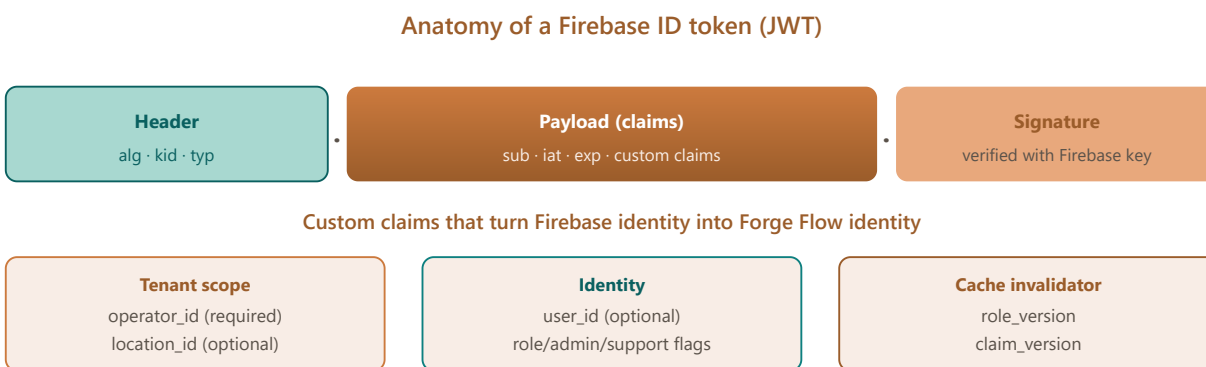


Figure 11.1. A Firebase ID token has three parts. The custom claims in the middle are what turn a Firebase user into a Forge Flow operator/location/role context. Missing required claims = fail closed.

When a sign-in succeeds, the proxy writes a row to the auth session ledger in Postgres so that the session has a server-side record beyond the device. The ledger captures login, refresh, and revoke events, which makes session lifecycle auditable and revocable. The app stores a secure session envelope in device-

protected storage so the session can be rehydrated without keeping plain credentials around.

How It Works

```
user enters email/password
-> Firebase validates credentials
-> app receives Firebase ID token
-> app/proxy requires Forge Flow custom claims
-> proxy writes auth session ledger in Postgres
-> app stores secure session envelope
-> app unlocks authenticated surfaces
```

WHERE IT LIVES

```
Auth state: lib/state/auth_session_notifier.dart . Runtime bindings:
lib/services/auth/firebase_auth_runtime_bindings.dart . Login service:
lib/services/auth/firebase_auth_login_service.dart . Proxy gateway:
lib/services/auth/proxy_auth_operations_gateway.dart . Auth schema:
db/migrations/202604250008_auth_schema_foundation.sql .
```

MFA And Admin Reset

TOTP at launch, admin-mediated recovery, and a 24-hour delay.

Multi-factor authentication means password plus another proof. The launch path uses TOTP, the rotating six-digit codes from an authenticator app, because it is strong, standard, and avoids SMS dependency. Recovery-code display and challenge entry are intentionally not exposed in the launch UX; the visible product path for a user who loses access to their authenticator is restaurant-admin reset with a 24-hour removal delay.

Setup, Recovery, And Throttling

TOTP setup runs through the proxy and Firebase's Identity Toolkit. The user begins setup, the app asks the proxy to start an enrollment, the proxy talks to the Identity Toolkit to create the TOTP secret, the user scans or types the secret into their authenticator, and the user confirms a code. The proxy records the MFA factor state in Postgres.

When a user loses their second factor, the recovery flow is admin-mediated. The user selects "Contact your admin," and a restaurant admin starts the reset from the Team surface. The proxy schedules a 24-hour removal that either the user or the admin can cancel while it is pending; when the window completes, a worker performs the actual removal and writes audit records. This delayed removal keeps account recovery tied to ownership and audit, and means the visible product never displays backup codes that could be stolen alongside a password.

An attempt ledger records MFA and recovery attempts to support rate-limit decisions and abuse detection, and rate limits throttle repeated guesses so that recovery and MFA endpoints cannot be hammered by automation.

Admin-mediated MFA reset: the 24-hour window

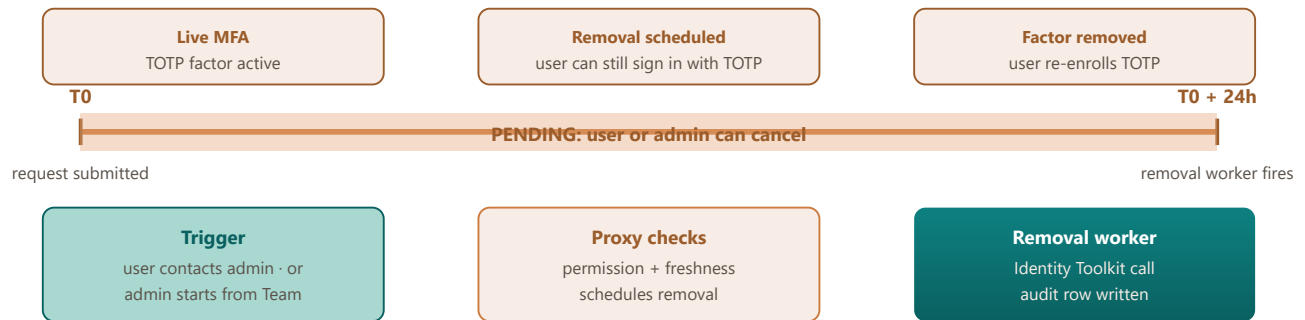


Figure 12.1. The 24-hour window. Recovery is admin-mediated and reversible until the worker fires. No backup codes are surfaced to the user, that path is intentionally invisible.

How It Works

TOTP setup:

```
user begins TOTP setup
-> app calls proxy
-> proxy talks to Firebase Identity Toolkit
-> user scans/setup secret
-> user confirms code
-> proxy records MFA factor state
```

Admin reset and delayed removal:

```
user requests reset or admin starts reset
-> app sends reset request to proxy
-> proxy checks freshness and permissions
-> proxy schedules 24-hour removal
-> user/admin may cancel while pending
-> worker completes the removal when due
-> audit records are written
```

WHERE IT LIVES

MFA gateway: `lib/services/mfa/proxy_mfa_operations_gateway.dart` . Recovery attempts schema: `db/migrations/202604280011_phase_9_0sigma_recovery_code_attempts.sql` . Proxy MFA routes: `tool/advisor_proxy/advisor_proxy.dart` .

Permissions, Roles, And Admin

Snapshots from the server, not guesses from the client.

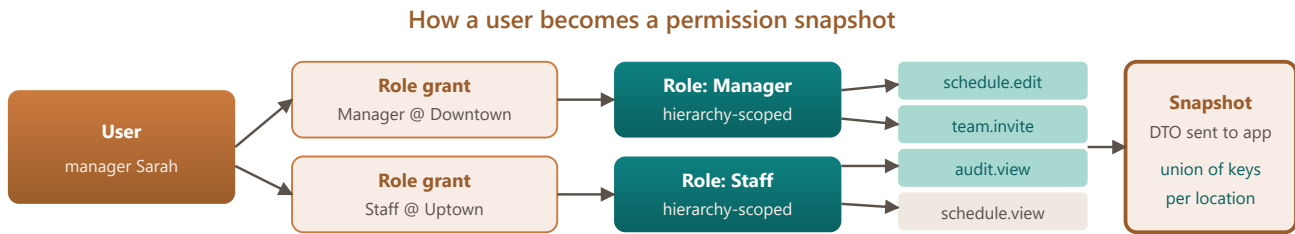
Permissions answer "What are you allowed to do?" Forge Flow expresses access as roles so capability sets are manageable. The app does not guess permissions from a job title, it asks for a permission snapshot and obeys it.

RBAC, Snapshots, And Admin Routes

The model is **role-based access control**: users get roles, and roles expand into a set of named permission keys such as `auth.roles.update` or `schedule.view`. Roles bundle permission keys, and a **role grant** attaches a role to a user inside an operator or location context, for example, "Sarah has Manager at Downtown." Roles are defined in a hierarchy-scoped way (per the May 2026 roles redesign in `db/migrations/202605142100_phase_R_1L_roles_schema_rewrite.sql`) so role definitions can inherit from business level down through org units to specific locations.

The **permission snapshot** is the resolved view that the app actually uses. When the app asks the proxy for `/v1/auth/permissions/snapshot`, the proxy verifies the user, reads the relevant role grants and role permissions, resolves the effective permission set, and returns it as a DTO. The UI then enables and disables protected actions based on that snapshot.

Administrative changes happen through protected `/v1/admin/auth/*` routes on the proxy. Every administrative change writes to a role audit log that preserves who changed access and when. The launch model is RBAC, but the architecture leaves room for **relationship-based access control** (ReBAC) in Phase 12.



Permission snapshot

*The UI never recomputes auth logic, it just toggles widgets against the keys in the snapshot.
role_version + claim_version invalidate the snapshot the moment access changes server-side.*

Figure 13.1. Role grants attach a user to a role within a scope; roles expand to permission keys; the snapshot is the resolved DTO the UI listens to.

How It Works

```

app requests permission snapshot
-> proxy verifies user
-> proxy reads roles and role permissions
-> proxy resolves effective permissions
-> app receives permission snapshot
-> UI enables/disables protected actions
  
```

WHERE IT LIVES

Role repositories: `lib/infrastructure/persistence/postgres/repositories/` . Auth schema:
`db/migrations/202604250008_auth_schema_foundation.sql` . Roles rewrite:
`db/migrations/202605142100_phase_R_1L_roles_schema_rewrite.sql` . Admin routes:
`tool/advisor_proxy/advisor_proxy.dart` .

Service Principals

Automation gets its own identity, never borrows a human's.

A service principal is a non-human identity. Humans log in with Firebase. Automation should not pretend to be a human. A workflow or backend job needs its own identity so audit logs can say exactly what acted, for example, the actor for a nightly audit anchor job is `sp:nightly-audit-anchor-job`, not the engineer who deployed it.

Issuance, Verification, And Audit Attribution

Service principals receive scoped credentials in the form of JWTs whose subjects are prefixed `sp:` (for example, `sp:rollup-worker`). The prefix makes it trivial for the proxy to distinguish automation tokens from Firebase human tokens. An **issuer** route inside the proxy creates the signed token for an approved service, and a **verifier** validates incoming `sp:` tokens before any work runs.

Every audit row stores an **actor kind** field so a reviewer can tell whether a person, a service, or the system acted. Actor-kind values are drawn from a single canonical catalog, `ActorKindLabelCatalog` in `lib/services/auth/actor_kind_label_catalog.dart`. Both the `user` and `team_member` values resolve to the display label **"Team member"**; both `service` and `service_principal` resolve to **"Service account"**. Both the admin console and the operator-web audit surface read labels from this single source, so the same audit row reads consistently regardless of which UI is rendering it.

How the proxy attributes every audit row, end to end

Who is acting?	Token shape	actor_kind enum value	Display label	Example
Human password + MFA login through Firebase	Firebase ID token custom claims: operator_id · user_id · roles_version	'user' or 'team_member'	"Team member" <i>in admin + operator-web audit</i>	<i>manager Sarah edits a schedule</i>
Automation workflow / worker / scheduled job	sp:-prefixed JWT issued + verified by proxy sub: "sp:rollup-worker"	'service' or 'service_principal'	"Service account" <i>in admin + operator-web audit</i>	<i>sp:rollup-worker refreshes rollups</i>
System backend maintenance not tied to a user	runAsSystem(reason) no JWT, proxy-internal requires explicit reason string	'system' (catalogue fallback)	"System" <i>distinguishable from automation</i>	<i>nightly audit anchor sweep</i>

Single source: `lib/services/auth/actor_kind_label_catalog.dart`

Both display surfaces (admin console, operator-web audit) read these labels from the same catalogue. Audit rendering can't drift between UIs.

Figure 14.1. One audit row, four columns: what kind of actor, what token they presented, what enum value the row gets, and what the audit UI shows. The catalogue at the bottom is the single source the labels resolve through.

How It Works

workflow needs to run

- > requests or receives service-principal JWT
- > proxy verifies `sp:` token
- > proxy derives service actor context
- > action runs with scoped permissions
- > audit log records service actor

WHERE IT LIVES

Service-principal schema: `db/migrations/202604280004_phase_9_0sigma_d_service_principals.sql` . JWT

issuance/verification: `tool/advisor_proxy/advisor_proxy.dart` . Actor-kind catalog:

`lib/services/auth/actor_kind_label_catalog.dart` .

Audit And Compliance

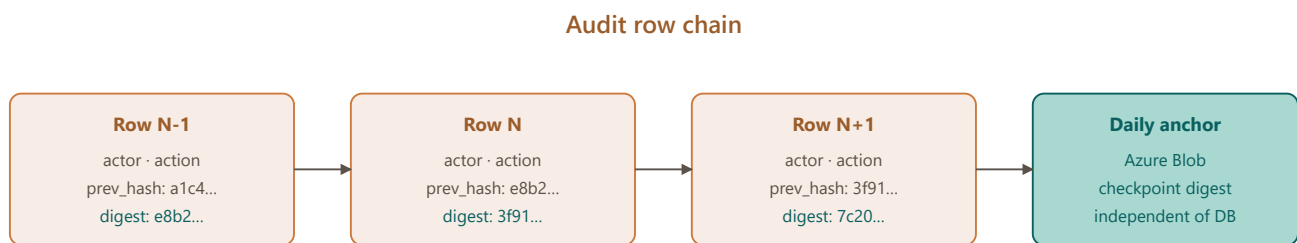
Tamper-evident logs: alteration becomes detectable, not impossible.

Audit answers "Who did what, when, and under which authority?" For sensitive systems, ordinary logs are not enough, they can be edited or rotated away. Forge Flow is building toward tamper-evident audit logs where records are chained together with cryptographic hashes, so that altering an old audit entry breaks the chain that follows it. This does not make the database impossible to alter; it makes alteration detectable.

Hash Chains, Anchors, And Redaction

Every sensitive action writes a durable **audit row** that captures the actor (human or service), the action, the resource, the operator and location context, and the timestamp. Tamper evidence comes from chaining. Each audit row includes a reference to the previous row's hash, and `pgcrypto` computes the digest for the current row over its own content plus that previous hash. The result is a hash chain: changing any historical row produces a different digest, which breaks the chain at every row after it.

A daily **anchor** writes a chain checkpoint to an external Azure Blob, outside the normal table write path, so that even if Postgres history were tampered with, an independent point of comparison still exists.



Editing Row N changes its digest, which breaks every row after it, and disagrees with the daily anchor.

Figure 15.1. The audit hash chain. Each row's digest depends on the previous row's digest, and a daily anchor stores a checkpoint outside the database.

Privacy obligations like GDPR right-to-erasure are handled through redaction patterns rather than deletion. Redaction removes sensitive personal content from an audit row while preserving its structural shape, so the audit trail and the chain remain intact while the user-identifying detail is gone.

How It Works

sensitive action happens

- > audit row is written
- > row includes previous hash reference
- > pgcrypto computes digest
- > daily anchor records chain checkpoint
- > verifier can later walk the chain

WHERE IT LIVES

Audit schema: `db/migrations/202604280005_phase_9_0sigma_f_audit_logs.sql` . Audit anchor tool: `tool/audit_anchor/audit_anchor.dart` . Verification runbook: `runbooks/audit_chain_verify_runbook.md` . Contract: `docs/contracts/audit_log_architecture_contract.md` .

Eventing And Real Time

Outbox first, NOTIFY second, durable rows beat ephemeral pings.

MENTAL MODEL

When something changes on the server, other parts of the system may need to know. The safe pattern is not "send a notification and hope", it is to save the real database change, save an event row, and let a worker deliver the event. The event row is durable; a notification is only a wake-up.

Outbox, Notify, And Workers

The pattern is the **transactional outbox**: alongside the business write, the same transaction writes a row into the `event_outbox` table. Because the business write and the event live in the same transaction, the event is durable the moment the change is committed, it cannot be lost even if a worker is down or the network is flaky.

Workers learn about new events through `LISTEN / NOTIFY`: when an outbox row is inserted, Postgres fires a notification that wakes any listening worker. The notification is just a wake-up; the worker reads the outbox table for the actual data, which means a missed notification is not a data loss. Workers claim rows for processing using `SELECT ... FOR UPDATE SKIP LOCKED`, which lets several workers run in parallel without two of them claiming the same row.

A **lease** model means each worker owns a row for a limited time. Consumers handle events **idempotently**: retrying the same event does not double-charge, double-write, or duplicate side effects. Events that repeatedly fail are moved to a **dead-letter** holding area. Downstream fan-out to **Pub/Sub** subscribers or **WebSocket**-connected clients lets multiple consumers receive the same change.

Postgres's `pgmq` extension is not available on Azure Flexible Server, which is why the system uses `SKIP LOCKED` (and, for some workloads, Cloud Tasks) rather than a managed queue.

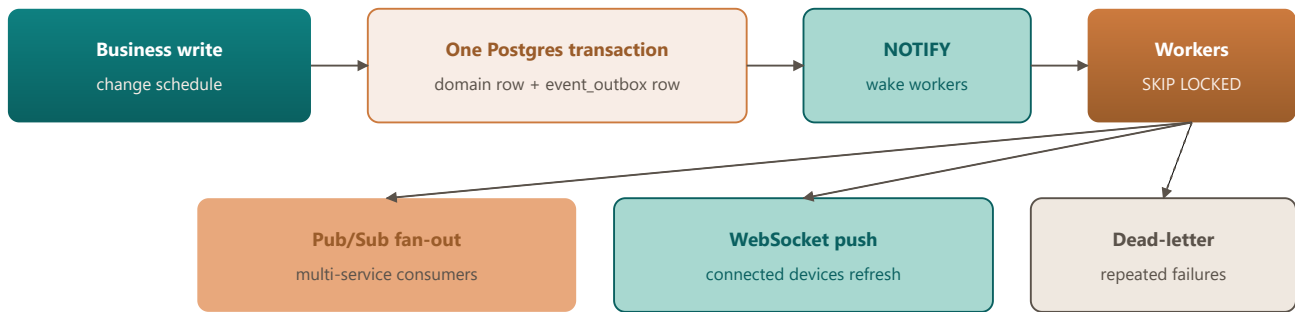


Figure 16.1. The transactional outbox makes events durable; NOTIFY only wakes workers, which claim rows with SKIP LOCKED and fan out to consumers.

How It Works

```
business mutation happens
-> event_outbox row is written
-> Postgres NOTIFY wakes worker
-> worker claims rows with SKIP LOCKED
-> worker publishes event
-> row is marked delivered
-> app refresh/invalidation can happen
```

WHERE IT LIVES

Event-outbox contract: `docs/contracts/event_outbox_contract.md` . Schema:
`db/migrations/202604280003_phase_9_0sigma_e_event_outbox.sql` .

Usage, Caps, And Rollups

Count consumption, enforce limits, summarize for dashboards.

Usage tracking answers "how much did someone use?" Caps answer "how much are they allowed to use?" Rollups answer "can we summarize a lot of raw detail into faster dashboard numbers?" Together they cover three jobs: counting consumption, enforcing limits, and pre-computing summaries so dashboards stay fast.

Logs, Caps, And Reconciliation

Every measurable consumption, an advisor call, a vendor sync, an export, writes a row to a **usage log** so that billing, abuse detection, and product limits all have a factual trail. **Usage caps** define how much usage is allowed by operator, slot, and period. The cap identity uses a **two-slot key model** that separates the dimensions necessary for reconciliation: an operator slot plus a location or capability slot. Reconciliation compares actual usage against the cap and surfaces overages, drift, or accounting mismatches before they turn into surprise bills.

Rollups are pre-computed summaries at multiple time grains, daypart, day, week, month, quarter, year, and at multiple entity levels, operator, location. Because dashboards do not have time to recompute totals from raw rows on every load, the rollup tables hold the answer and the dashboards read from them. An **aggregation state** tracker remembers when each rollup was last refreshed. The rollup refresh work runs as scheduled Postgres jobs through the **pg_cron** extension.

How It Works

```
raw usage or operational events arrive
-> usage_logs record details
-> usage_caps define limits
-> rollup functions aggregate detail
-> dashboards read summary tables
-> health checks monitor freshness
```

The originally-tracked gap (item B33) for `usage_logs` needing a matching two-slot mirror for cap-versus-actual reconciliation is now complete: the writer and constraint posture support reconciliation.

WHERE IT LIVES

Usage foundation: `db/migrations/202604250005_advisor_cloud_foundation.sql` . Two-slot cap migrations: `db/migrations/202604280006_a/b/c_phase_9_0sigma_g_usage_caps_two_slot_*.sql` . Rollup migrations: `db/migrations/202604280010_a/b/c_phase_9_0sigma_k_rollups_*.sql` .

The Advisor And AI Layer

Evidence first, then synthesis, and recommendations only, never actions.

The advisor is not just "ask a chatbot." It is supposed to answer from evidence, product docs, operational metrics, SQL data, graph relationships, prior context, and explain why. The advisor recommends, explains, and cites the context that produced the answer, and it does not silently perform risky actions. Forge Flow's sixth Hard Promise binds this: the advisor speaks in recommendations, not commands, and never acts on the operator's behalf at launch.

Providers, Retrieval, And Synthesis

The architecture treats every external AI dependency through a **provider abstraction**. The launch providers are Anthropic Claude for synthesis and classification, and Voyage for embeddings and reranking; the same `LLMProvider`, `EmbeddingProvider`, `RerankProvider`, `DataSourceProvider`, and `IntegrationProvider` interfaces will be reused by Phase 12.

When a user asks an operational question, a **classifier** decides the route. Once the path is chosen, retrieval-augmented generation (**RAG**) collects evidence: dense **vector search** over **pgvector** embeddings finds documents close to the question, sparse full-text search adds keyword precision, and AGE-based graph queries pull related entities and edges. A **reranker** model, Voyage's `rerank-2.5`, re-sorts the retrieved candidates so the most useful evidence comes first.

Synthesis is the final answer generation, produced by Anthropic Claude Sonnet. **Tool use** lets the model invoke structured calls. Every answer carries **provenance**: the rows, documents, and graph edges that supported the answer are recorded so a user can inspect why a recommendation was made. **Prompt caching** reuses stable system context across calls to lower cost and latency, and **model routing** sends easy tasks to Claude Haiku and harder tasks to Claude Sonnet.

Routing keeps cost and quality on the right side of the margin

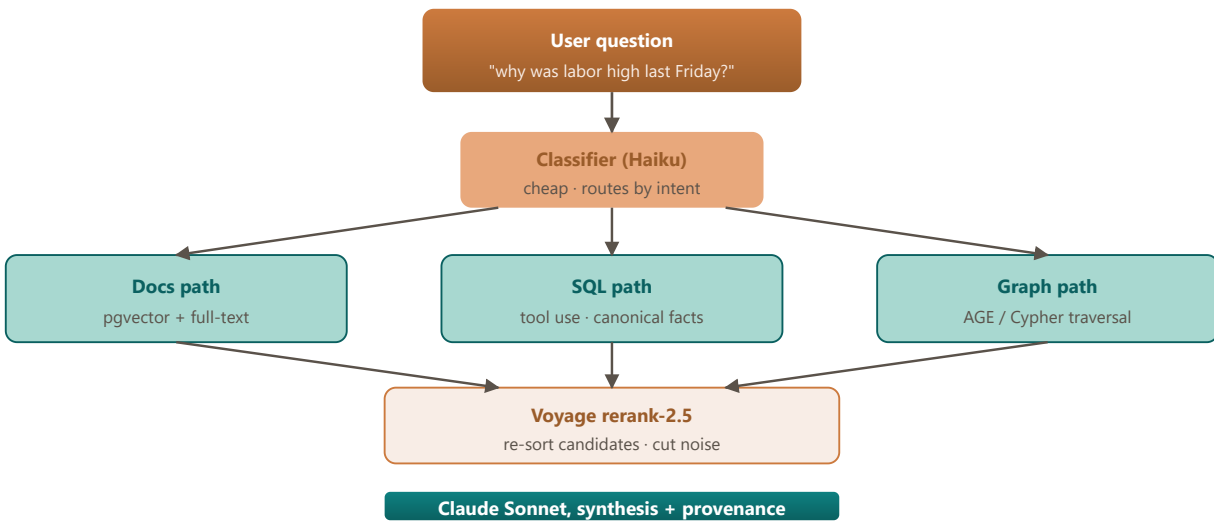


Figure 18.1. A cheap classifier picks the route; heavy work happens only on the chosen path. Voyage rerank reduces noise before Sonnet synthesises the final answer with citations.

How It Works

- user asks a question
- > classifier decides the route
- > system retrieves docs, SQL facts, graph facts, or mixed evidence
- > Voyage reranks evidence
- > Claude synthesizes answer
- > app shows recommendation and supporting context

WHERE IT LIVES

Provider interfaces: `lib/domain/services/llm_provider.dart`, `embedding_provider.dart`, `rerank_provider.dart`. Primary LLM: `lib/services/claude_llm_provider.dart`. Fallback LLM: `lib/services/gemini_llm_provider.dart` (Block 2 fallback). Embeddings + rerank: `lib/services/voyage_embedding_provider.dart`, `voyage_rerank_provider.dart`. Decisions: `docs/phases/phase_11a/phase_11a_decision_register.md`.

The Graph Layer

Tables are for rows. Graphs are for relationships.

Tables are good for rows. Graphs are good for relationships. The graph layer helps Forge Flow answer questions where the connections matter: what caused this, what is related to this issue, which document or workflow or metric connects to this symptom. Forge Flow's fifth Hard Promise commits to AGE infrastructure being live before Phase 11b, with retrieval growing into Modular Adaptive Agentic RAG over time.

Nodes, Edges, And Traversal

A **graph** is a network of connected things, made up of **nodes** that represent entities (a location, a shift, a vendor, a metric, a document, a workflow concept) and **edges** that represent typed relationships ("causes," "references," "belongs to," "depends on"). Inside Postgres, the **AGE** extension provides property-graph capabilities, and **Cypher** is the query language that expresses traversals directly instead of forcing relational joins to mimic them.

The graph is not a competing source of truth: it is a **projection** built from canonical facts. When source data changes, the graph view is rebuilt from canonical truth, which means the graph always reflects the same operational reality the rest of the system uses. **Tripwire metrics** monitor node count, edge count, and degree distribution so growth that would make traversal unhealthy is visible early.

How It Works

```
canonical facts/docs are processed
-> graph nodes represent entities/concepts
-> graph edges represent relationships
-> advisor or health tools query relationships
-> system can explain connected causes/evidence
```

WHERE IT LIVES

Graph schema: `db/migrations/202604280008_phase_9_0sigma_i_graph_canonical.sql` . Future consumers include Phase 11b advisor traversal and Phase 11A health.

Live Integrations

Adapters, not rewrites, vendor data enters through one shape.

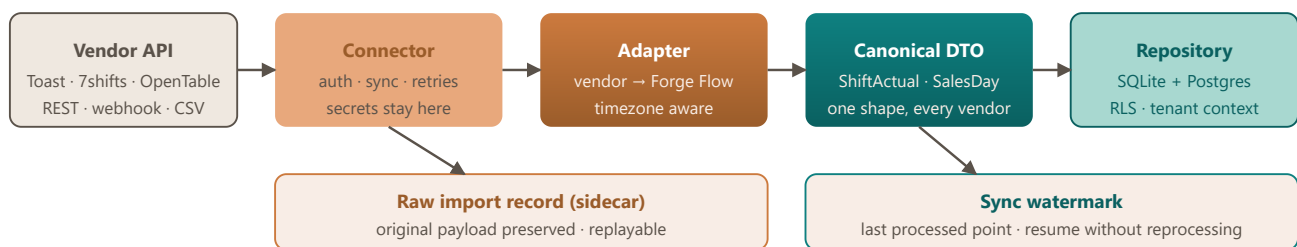
Forge Flow should eventually use live vendor data from POS, labor, reservation, accounting, and banking systems, but the UI should not be rewritten for every vendor. Vendor data enters through adapters, becomes canonical Forge Flow facts, and then flows through the same screens, the same advisor, and the same audit and tenant safety as anything else. Hard Promise #1 makes this explicit: Phase 8 is a pure transport swap.

Connectors, Adapters, And Watermarks

A **connector** owns the conversation with one vendor: API calls, authentication, sync orchestration, webhook handling, and rate-limit behavior. An **adapter** then translates the vendor-specific payload into a **canonical DTO**: the Forge Flow shape that the rest of the product is built against, so vendor-specific differences stay at the boundary and never leak into screens or services.

Vendor **secrets** stay server-side: API keys, OAuth tokens, and refresh credentials live in Secret Manager and are reached only by the proxy and connector code. **OAuth** is the delegated authorization flow. **Webhooks** push near-real-time updates from the vendor when something changes, and polling with **sync watermarks** handles pull-based reads. Every vendor payload is also stored as a **raw import record** before adaptation, so if the mapping logic changes later the original evidence can be inspected or replayed.

Vendor data is shaped at the boundary, never inside the product



UI and advisor only ever see the canonical DTO, vendor mess never crosses the boundary.

Figure 20.1. Five pipeline stages plus two sidecars. Raw payloads are kept for replay; watermarks make sync incremental.

How It Works

```
vendor API provides payload
-> trusted connector fetches/receives data
-> adapter converts payload into canonical facts
-> repository stores facts
-> SQLite/Postgres/state update
-> UI and advisor use the same internal model
```

WHERE IT LIVES

Vendor integration code: `lib/integrations/`. Integration spine contract:

`docs/contracts/integration_spine_architecture_contract.md`. Phase plans:

`docs/archive/phases/phase_8/`, `phase_8R/`, `phase_8_5_external_integrations/`.

Shared State Roadmap

Truth on the server, fast caches on the device, events in between.

Shared state means multiple devices can see and update the same server-backed data. SQLite continues to make the local app fast, but Postgres becomes the source of truth for shared data, the proxy controls writes, and events tell other devices to refresh.

Truth, Cache, And Invalidation

The model is straightforward: Postgres holds the **source of truth**, and SQLite holds a **cache** that makes reads fast and tolerant of network hiccups. When something changes on the server, an **invalidation** signal tells connected devices that their local cache for the affected entity is stale; the device fetches the new version rather than full-reloading every screen. **Startup sync** runs when a device first opens or returns to the app, hydrating the local cache from the server.

The path between server-side change and device-side refresh runs through an **event bridge**: a transactional outbox row, a worker, and a delivery channel (Pub/Sub fan-out or WebSocket push). When two clients edit the same shared entity, Forge Flow's V1 conflict policy is **last-write-wins**. Every shared mutation is audited so that "who changed this" is always answerable.

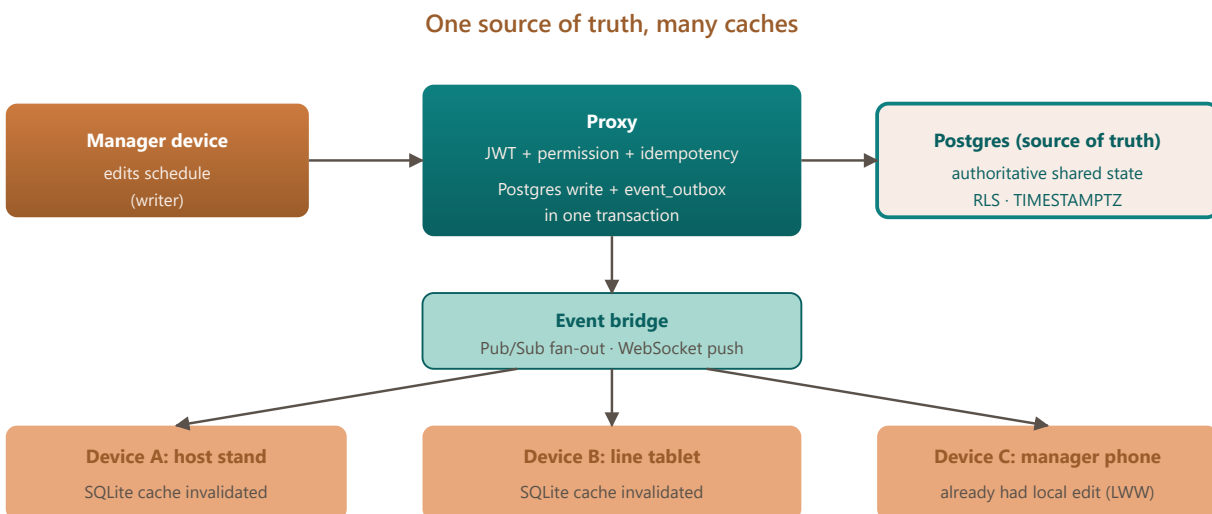


Figure 21.1. One device writes, every other device learns its cache is stale. V1 conflict policy is last-write-wins.

How It Works

```
manager changes shared setting
-> app calls proxy
-> proxy verifies JWT and permission
-> Postgres write runs with RLS
-> event_outbox records change
-> worker/bridge notifies clients
-> other devices refresh SQLite cache
```

WHERE IT LIVES

Shared state plan: `docs/archive/phases/phase_10a/phase_10a_shared_state_v1_plan.md` . Event contract: `docs/contracts/event_outbox_contract.md` .

The Operations Console

A staff-only Flutter Web surface that calls protected admin APIs.

The operations console is the internal admin tool that Forge & Flow staff use to manage the platform itself, not the restaurant operator app. It is where the team manages operators, locations, pricing, feature flags, the advisor corpus, integrations, support actions, audit reviews, and health monitoring.

Internal Admin Surface

The internal admin app is a Flutter Web surface that calls protected **admin API** routes on the proxy. Every administrative action goes through the proxy, which means admin power stays behind server-side identity, MFA, permission, and audit checks rather than relying on direct database access.

Feature flags let the team enable or disable capabilities at runtime, supporting staged rollout and quick rollback without shipping a new app build. **Observability** surfaces, health checks, logs, metrics, and the bounded dashboard delivered in Phase 11A.6, let staff diagnose problems before users report them. **Support actions** are the named, audited operations that resolve customer issues so support becomes repeatable rather than ad-hoc.

The hard line is that the browser never receives direct database access. The ops console calls API routes, and the proxy enforces every check; a compromised browser session cannot become raw Postgres access.

How It Works

```
internal admin signs in
-> admin console calls /v1/admin/*
-> proxy verifies admin identity and MFA
-> proxy checks admin permission
-> proxy performs scoped server action
-> audit log records the action
```

WHERE IT LIVES

Admin console code: `lib/admin/`. Ops console plan:

`docs/phases/phase_11A_operations_console/phase_11A_operations_console_plan.md`. Admin routes:
`tool/advisor_proxy/advisor_proxy.dart`.

The Workflow Platform

Powerful, but never invisible: identity, approvals, costs, and audit.

The workflow platform is the future automation layer. It will let Forge Flow run scheduled or event-driven work, but with strict identity, permission, audit, cost, and approval controls. The safety idea is that automation should be powerful, but never invisible.

Triggers, Tools, And Approval Gates

A **workflow** is a defined sequence of steps that runs in response to a **trigger**. The workflow authenticates as a service principal (chapter 14), which is what keeps its actions distinct from human actions in the audit log. The **tool registry** is the list of allowed actions a workflow may call; nothing else is reachable.

The execution model is **Plan-Then-Execute**: the workflow proposes a plan first, and only after that plan exists does it execute. Risky steps wait at an **approval gate** for a human to review and approve before any state changes. Outputs are preserved so a run is reviewable after the fact. Long or batch work runs as a **Cloud Run job**. **Cost caps** stop runaway provider spend, and **batch APIs** route non-urgent AI work to cheaper asynchronous processing.

How It Works

```
trigger occurs
-> workflow service principal authenticates
-> workflow creates plan
-> risky steps wait for approval
-> approved tools execute
-> events/audit records are written
-> artifacts are stored if needed
```

WHERE IT LIVES

Workflow plan:

`docs/archive/phases/phase_12_workflow_platform/phase_12_workflow_platform_plan.md` . Service-principal foundation: `db/migrations/202604280004_phase_9_0sigma_d_service_principals.sql` .

Deployment, Secrets, And Environments

Local for dev, staging for proof, production1 behind gates.

An environment is a place the system runs. Local is for development. Staging is for testing real cloud wiring before production. Production1 is the protected real environment and only changes through explicit runbooks and gates. Secrets are sensitive values that must never be committed to Git or baked into the Flutter app bundle.

Environments, Secrets, And Releases

Local runs the app and supporting services on a developer machine. **Staging** proves the real cloud wiring: DNS, certificates, Cloud Run, the proxy, the secret store, and smoke tests. **Production1** is the customer-facing environment, where changes flow only through documented runbooks and explicit gates, destructive operations, force pushes to main, RLS-touching migrations, schema-touching migrations, proxy-touching slices, and ceiling raises like `kAdvisorProxyMaxLines` all require explicit operator approval.

Secrets live in Google **Secret Manager**, and Cloud Run reads them as secret references at startup. **Environment variables** carry non-secret runtime configuration. The proxy is built from a **Dockerfile** that describes the exact runtime environment in code. Continuous integration runs on **GitHub Actions**. High-risk live operations follow **runbooks** (consolidated into a single `runbooks/` home as of the 2026-05-16 Wave 4 docs consolidation). A **Google-managed certificate** secures public HTTPS without manual renewal.

Same code, three environments: what each one is allowed to touch

Resource / capability	Local	Staging	Production1
Real customer data	X, never	X, synthetic only	✓
Database	docker-compose Postgres	Azure (staging tier)	Azure Canada Central
Firebase project	sandbox	staging tenant	prod tenant
Provider keys (Claude / Voyage / Gemini)	none / mocked	staging keys	prod keys (Secret Manager)
Public hostname	localhost	staging-api.feflow.org	api.feflow.org
Cloud Armor edge policy	n/a	monitor / log	monitor / log → enforce
Audit hash chain	in-memory / disposable	writes + verify	writes + daily Azure Blob anchor
What gates a change?	none, local rebuild	CI green	CI + operator approval + runbook

Same container, same migrations, same code.

The only differences between Local, Staging, and Production1 are the secret references (Secret Manager), the database connection string, and which gates a change.

Figure 24.1. Local, Staging, and Production1 are the same container with different secrets and different gates. The matrix shows what each environment is allowed to touch, and why a production change costs operator approval plus a runbook.

How It Works

A staging deploy looks like:

```
local private secrets file
-> deploy script validates required values
-> secrets sync to Google Secret Manager
-> Cloud Run deploy uses secret references
-> load balancer routes public traffic
-> smoke test verifies /readyz
```

A production migration apply looks like:

```
confirm gates
-> backup
-> apply migrations in documented order
-> run verification queries
-> monitor
-> use backout notes if needed
```

WHERE IT LIVES

Deploy script: `scripts/deploy_staging_proxy.ps1` . Proxy build: `Dockerfile` . Local Postgres scaffold: `docker-compose.dev.yml` . Production apply runbook: `runbooks/phase_9_production1_migration_apply_runbook.md` .

Testing And Gates

Static analysis, lint, tests, and smoke probes, safety on every layer.

Tests are safety rails. In this repo, tests protect more than UI, they protect auth, MFA, tenant isolation, database migration shape, proxy behavior, platform builds, and production readiness. Tests are not the only gate: static analysis, RLS lint, contract tests, migration tests, and smoke tests all sit alongside the test suite.

The Verification Layers

Static analysis runs `flutter analyze --fatal-infos` to catch Dart issues before the app ever runs. **Unit tests** check small, focused behaviors. **Widget tests** check Flutter UI behavior without booting a full device. **Contract tests** verify that the running code still obeys documented architectural shapes. **Migration tests** verify that schema changes create the indexes, constraints, and policies they promise. **RLS lint** at `tool/rls_policy_lint.dart` blocks unsafe RLS policy patterns automatically.

Smoke tests verify that a deployed service actually responds. **GitHub Actions** runs all of this on every push and pull request. A separate Apple verification workflow runs on a macOS runner and produces iOS simulator builds for both flavors.

What each verification gate catches, and how long you wait for the answer

Gate	What it catches	When it runs	Feedback time
flutter analyze --fatal-infos static analysis	type errors, dead code, deprecation, lint rules	every commit	~ 30 s
tool/rls_policy_lint.dart RLS shape check	bare current_setting() in policies; index lead column	every commit	~ 5 s
flutter test (unit) unit tests	labor variance, formula correctness, DTO mapping	every commit	~ 2 min
flutter test (widget) widget tests	UI regressions without booting a device	every commit	~ 2 min
contract + migration tests architecture drift	RLS policies, index shape, hash-chain wiring promises	every commit	~ 1 min
apple-platform-verify iOS build / sim	iOS-specific compile breaks; ForgeFlow + Barrio flavors	every PR	~ 10 min
GET /readyz smoke live wiring proof	deployment + DNS + cert + load balancer + Cloud Run all reachable	on deploy	~ 5 s
Runtime acceptance (reviewer judgment) advisory · not CI-enforced	"does this actually behave correctly when the operator clicks?"	on review	human time

Gates run fastest-first so the cheapest answers come back before the expensive ones. The advisory gate is the only one a human still has to do, see `slice_runtime_acceptance_contract.md`.

Figure 25.1. Eight gates, ordered by feedback speed. Each row says what it catches, when it runs, and how long the wait is. The dashed row at the bottom is the only one CI can't enforce.

How It Works

```
push or pull request
-> GitHub Actions starts
-> flutter pub get
-> flutter analyze --fatal-infos
-> dart run tool/rls_policy_lint.dart
-> flutter test
```

WHERE IT LIVES

Main CI: `.github/workflows/ci.yml` . Apple verify: `.github/workflows/apple-platform-verify.yml` . Tests: `test/` . RLS lint: `tool/rls_policy_lint.dart` .

Current Status And Known Gaps

A snapshot of what is live, scaffolded, or still planned.

The foundation is broad, and much of it is now merged or scaffolded. Some database structures exist before their final UI, worker, or workflow consumer exists, and that is normal for this repo's phase style. The doctrine is: build the foundation, verify the contracts, then attach the consumer features.

Snapshot

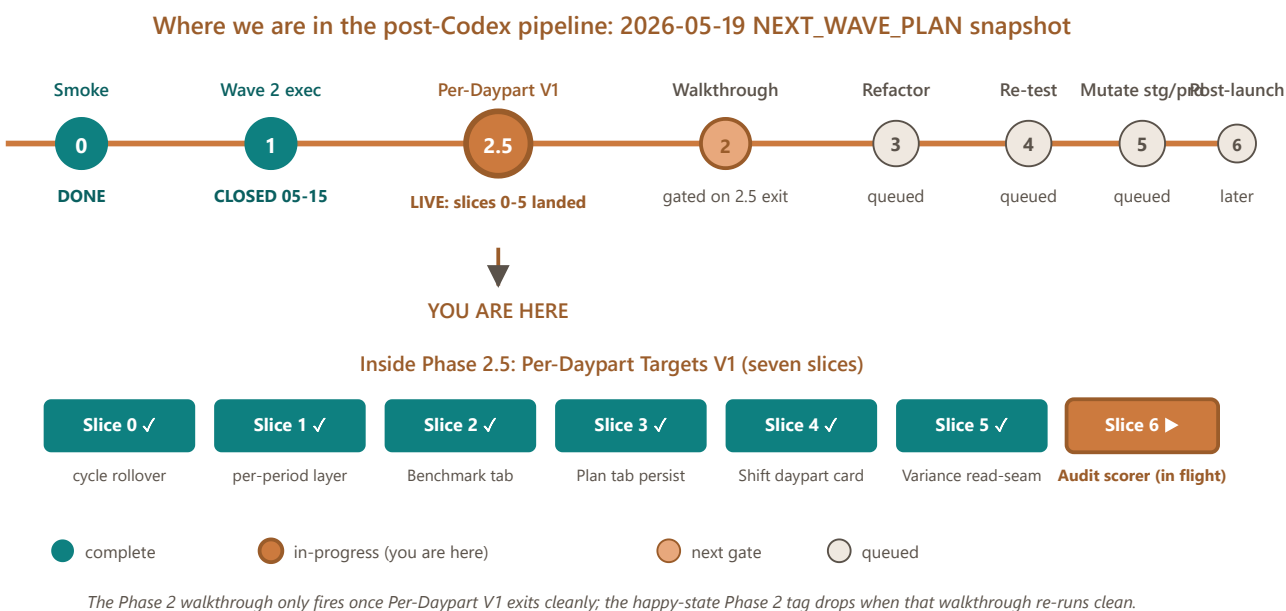


Figure 26.1. The pipeline at the top + a zoom into the active phase below. Per-Daypart Targets V1 is mid-flight at Slice 6; the walkthrough phase is gated until V1 exits. Source: [docs/_indices/NEXT_WAVE_PLAN.md](#) status snapshot 2026-05-19.

The Phase 9.oSigma.b through k foundation is **Live**: auth, RLS, service-principal, audit, event, graph, and rollup groundwork is on master. Staging DNS and HTTPS are passing. Phase 9 is accepted: live auth closeout is accepted for next-phase handoff, Cloud Armor is in monitored preview, and physical iOS QA is deferred. The Phase 9 migrations through [202604280013](#) are applied to Production1.

The post-Codex execution pipeline is mid-flight. As of the 2026-05-19 [NEXT_WAVE_PLAN.md](#) status snapshot: Phase 0 (smoke) is complete; Phase 1 (Wave 2) is CLOSED (Wave 1 closed 2026-05-13; Wave 2 ran 33 slices across 11 lanes with operator-web + admin lanes closed 2026-05-14 and mobile closed 2026-05-15). **Phase 2.5 (Per-Daypart Targets V1) is the live phase**, recording seven slices (Slice 0

through Slice 6). **Slices 0 through 5 have landed; Slice 6 and any later follow-ups are still in flight.** The happy-state Phase 2 tag drops only when Phase 2.5 exits and the post-Wave-2 walkthrough re-runs cleanly. The team currently runs dual-Claude orchestration (main orchestrator + a second Claude lane) because Codex is out of quota.

Recent landings: the hierarchy-scoped roles redesign (`202605142100`) and follow-ups, hierarchy-scoped benchmark overrides (`202605131550`), self-service profile permission key (`202605140000`), email-event provider id (`202605131700`), and the Per-Daypart V1 migration chain (`202605160000` through `202605170200`). The admin actor-kind label unification (2026-05-14) brought admin and operator-web audit surfaces onto a single canonical catalog. The Wave 4 docs consolidation (2026-05-16) unified runbooks into one home.

The B17 admin role catalog CRUD is `Live` and smoke-passed on staging. The B33 `usage_logs` two-slot mirror work is `Live`. The B34 audit attribution contract clarification is `Live`. The RLS isolation sweep is `Live`. The service-principal issuer route is `Live in schema`. The event-outbox is `Scaffolded` in Phase 10a; Pub/Sub adapter, dead-letter handling, retention sweep, tripwires, and operator-facing UX remain queued. Health producers B44 (graph), B45 (rollup), and B47 (vector) now fill the B42 health envelope.

The runtime user-facing demo↔live switch landed in `lib/screens/settings/settings_demo_live_switch.dart` in May 2026, so an operator can flip demo→live state at runtime without a rebuild, while the demo-mode reader path remains a single, non-branching code path per Hard Promise #2.

WHERE IT LIVES

Current tracker: `PROJECT_TRACKER.md`. Forward plan: `docs/_indices/NEXT_WAVE_PLAN.md`. Active feature plan: `docs/phases/per_daypart_targets_v1/per_daypart_targets_v1_plan.md`. Production apply runbook: `runbooks/phase_9_production1_migration_apply_runbook.md`.

Quick Glossary

A compact reference for the most-used terms.

G

A record

A DNS entry that points a hostname to an IPv4 address; `staging-api` points to `34.54.204.29`.

AGE

The Postgres extension that adds property-graph capability to relational tables.

API

A server route or interface that software calls; `GET /readyz` and `POST /v1/auth/session/login` are examples.

Business date

The restaurant-local operating date that anchors reporting; a 1 AM close still belongs to Friday service.

CNAME

A DNS entry that points one hostname to another hostname.

Cloud Armor

Google's edge security layer that logs or blocks suspicious requests before they reach the backend.

Cloud Run

The Google service that runs containers; the staging proxy runs there as `forge-flow-staging-proxy`.

DNS

The internet's naming system that translates hostnames into network destinations.

Firebase Identity Platform

Google's hosted login, auth, and MFA service.

Hostname

A friendly service name like `staging-api.feflow.org`.

JWT

A signed identity token; the Firebase ID token the app sends to the proxy is one.

Load balancer

The public traffic router that terminates HTTPS and forwards requests.

MFA

More than one proof of identity (password plus an authenticator code).

NEG

A serverless network endpoint group that connects a Google load balancer to Cloud Run.

Pgvector

The Postgres extension that stores and similarity-searches embeddings.

Proxy

The server-side API gatekeeper that holds secrets, verifies identity, and performs privileged work.

RAG

Retrieval-Augmented Generation: retrieve evidence first, then have the model generate an answer.

RLS

Postgres Row Level Security, which hides rows that do not match the current tenant context.

Secret Manager

Google's storage for sensitive configuration values.

Service principal

A non-human automation identity (for example `sp:rollup-worker`).

SQLite

The local embedded database the app uses for caches, replay data, and demo state.

TOTP

A rotating time-based one-time password produced by an authenticator app.

WebSocket

A long-lived real-time connection between the server and a client.

Source Documents And Code Read

S

Every contract, runbook, and code path consulted while writing this guide.

This guide was synthesized from `README.md`, `PROJECT_TRACKER.md`, `CLAUDE.md`, `pubspec.yaml`, `Dockerfile`, and `docker-compose.dev.yml` at the repo root; the contracts directory including `docs/contracts/core_app_architecture.md` (canonical Phase 7.55 architecture), `phase_7_55_plain_english_architecture.md`, `event_outbox_contract.md`, `demo_mode_contract.md`, `audit_log_architecture_contract.md`, `hardening_ribs_and_repository_pattern_contract.md`, and `integration_spine_architecture_contract.md`; the active feature plan `docs/phases/per_daypart_targets_v1/per_daypart_targets_v1_plan.md`; the forward plan `docs/_indices/NEXT_WAVE_PLAN.md`; the historical phase plans for Phases 8, 8R, 8.5, 9, 10a, 11a, 11A, and 12; the consolidated runbooks under `runbooks/`; and the code at `lib/main_forgeflow.dart`, `lib/main_barrio.dart`, `lib/forged_flow_bootstrap.dart`, `lib/forged_flow_app.dart`, `lib/state/auth_session_notifier.dart`, `lib/services/auth/firebase_auth_runtime_bindings.dart`, `lib/services/auth/firebase_auth_login_service.dart`, `lib/services/auth/actor_kind_label_catalog.dart`, `lib/services/mfa/proxy_mfa_operations_gateway.dart`, `lib/screens/settings/settings_demo_live_switch.dart`, `lib/admin/`, `lib/operator_web/`, `lib/integrations/`, `lib/internal/`, `lib/infrastructure/persistence/sqlite/`, `lib/infrastructure/persistence/postgres/`, `tool/advisor_proxy/`, `db/migrations/`, `.github/workflows/ci.yml`, and `.github/workflows/apple-platform-verify.yml`.